

□ Unemployment Analysis in India

Data Science Internship Project — NumPy & Pandas

Unemployment is measured as the % of unemployed people out of the total labour force.

This project analyses how unemployment varied across Indian states from **May 2019 to June 2020**

— including the devastating impact of the **COVID-19 pandemic lockdown**.

□ Table of Contents

1. [Import Libraries](#)
2. [Load & Inspect Dataset](#)
3. [Data Cleaning & Preprocessing](#)
4. [Pandas EDA — Descriptive Statistics](#)
5. [NumPy Statistical Analysis](#)
6. [Region-wise Analysis](#)
7. [Rural vs Urban Analysis](#)
8. [COVID-19 Impact Analysis](#)
9. [Correlation Analysis](#)
10. [Visualizations](#)
11. [Key Insights & Conclusions](#)

1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

# Plot style
sns.set_theme(style="darkgrid", palette="muted")
plt.rcParams.update({
    "figure.dpi" : 110,
    "figure.figsize" : (12, 5),
    "axes.titlesize" : 13,
    "axes.titleweight" : "bold",
    "axes.spines.top" : False,
    "axes.spines.right" : False,
```

```

}))

print("Libraries loaded successfully ")
print(f" NumPy : {np.__version__}")
print(f" Pandas : {pd.__version__}")

Libraries loaded successfully 
NumPy : 2.4.3
Pandas : 3.0.1

```

2. Load & Inspect Dataset

```

df_raw = pd.read_csv("unemployementinindia.csv")
df_raw.columns = df_raw.columns.str.strip()

```

```

print(f"Shape : {df_raw.shape}")
print(f"Columns : {list(df_raw.columns)}")
df_raw.head(8)

```

```

Shape : (768, 7)
Columns : ['Region', 'Date', 'Frequency', 'Estimated Unemployment Rate (%)', 'Estimated Employed', 'Estimated Labour Participation Rate (%)', 'Area']

```

	Region	Date	Frequency	Estimated Unemployment Rate (%) \
0	Andhra Pradesh	31-05-2019	Monthly	3.65
1	Andhra Pradesh	30-06-2019	Monthly	3.05
2	Andhra Pradesh	31-07-2019	Monthly	3.75
3	Andhra Pradesh	31-08-2019	Monthly	3.32
4	Andhra Pradesh	30-09-2019	Monthly	5.17
5	Andhra Pradesh	31-10-2019	Monthly	3.52
6	Andhra Pradesh	30-11-2019	Monthly	4.12
7	Andhra Pradesh	31-12-2019	Monthly	4.38

	Estimated Employed	Estimated Labour Participation Rate (%)	Area
0	11999139.0	43.24	Rural
1	11755881.0	42.05	Rural
2	12086707.0	43.50	Rural

3	12285693.0	43.97	Rural
4	12256762.0	44.68	Rural
5	12017412.0	43.01	Rural
6	11397681.0	41.00	Rural
7	12528395.0	45.14	Rural

```
print("-" * 50)
print("DataFrame Info:")
print("-" * 50)
df_raw.info()
print("\nMissing values:")
print(df_raw.isnull().sum())
```

DataFrame Info:

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	Region	740 non-null	str
1	Date	740 non-null	str
2	Frequency	740 non-null	str
3	Estimated Unemployment Rate (%)	740 non-null	float64
4	Estimated Employed	740 non-null	float64
5	Estimated Labour Participation Rate (%)	740 non-null	float64
6	Area	740 non-null	str

```
dtypes: float64(3), str(4)
```

```
memory usage: 42.1 KB
```

Missing values:

Region	28
Date	28
Frequency	28
Estimated Unemployment Rate (%)	28
Estimated Employed	28
Estimated Labour Participation Rate (%)	28
Area	28

```
dtype: int64
```

3. Data Cleaning & Preprocessing

Steps:

- Rename columns to readable names
- Strip whitespace from strings
- Parse Date → datetime
- Convert numeric columns
- Drop null rows
- Engineer: Year, Month, Quarter, Period

```
df = df_raw.copy()

# Rename
df.columns = ["Region", "Date", "Frequency", "Unemployment_Rate",
              "Employed", "Labour_Participation_Rate", "Area"]

# Strip strings
for col in df.select_dtypes("object").columns:
    df[col] = df[col].str.strip()

# Parse date
df["Date"] = pd.to_datetime(df["Date"].str.strip(), format="%d-%m-%Y")

# Numeric
for col in ["Unemployment_Rate", "Employed", "Labour_Participation_Rate"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Drop nulls
df.dropna(subset=["Unemployment_Rate", "Employed", "Labour_Participation_Rate"],
          inplace=True)
df.reset_index(drop=True, inplace=True)

# Time features
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["MonthName"] = df["Date"].dt.month_name()
df["Quarter"] = df["Date"].dt.quarter

# COVID period label
df["Period"] = df["Date"].apply(
    lambda d: "During COVID" if d >= pd.Timestamp("2020-04-01") else
    "Pre-COVID"
)

print(f"Clean shape : {df.shape}")
print(f"Date range : {df['Date'].min().date()} → {df['Date'].max().date()}")
print(f"Regions : {df['Region'].nunique()}")
print(f"Area types : {df['Area'].unique()}")
df.head()
```

```

Clean shape : (740, 12)
Date range  : 2019-05-31 → 2020-06-30
Regions     : 28
Area types  : <StringArray>
['Rural', 'Urban']
Length: 2, dtype: str

```

	Region	Date	Frequency	Unemployment_Rate	Employed
0	Andhra Pradesh	2019-05-31	Monthly	3.65	11999139.0
1	Andhra Pradesh	2019-06-30	Monthly	3.05	11755881.0
2	Andhra Pradesh	2019-07-31	Monthly	3.75	12086707.0
3	Andhra Pradesh	2019-08-31	Monthly	3.32	12285693.0
4	Andhra Pradesh	2019-09-30	Monthly	5.17	12256762.0

Quarter \	Labour_Participation_Rate	Area	Year	Month	MonthName	
0	43.24	Rural	2019	5	May	2
1	42.05	Rural	2019	6	June	2
2	43.50	Rural	2019	7	July	3
3	43.97	Rural	2019	8	August	3
4	44.68	Rural	2019	9	September	3

	Period
0	Pre-COVID
1	Pre-COVID
2	Pre-COVID
3	Pre-COVID
4	Pre-COVID

4. Pandas EDA — Descriptive Statistics

```

# Descriptive statistics
print("=" * 55)
print("  Descriptive Statistics")
print("=" * 55)
df[["Unemployment_Rate", "Employed", "Labour_Participation_Rate"]].describe().round(2)

```

Descriptive Statistics

	Unemployment_Rate	Employed	Labour_Participation_Rate
count	740.00	740.00	740.00
mean	11.79	7204460.03	42.63
std	10.72	8087988.43	8.11
min	0.00	49420.00	13.33
25%	4.66	1190404.50	38.06
50%	8.35	4744178.50	41.16
75%	15.89	11275489.50	45.50
max	76.74	45777509.00	72.57

Groupby: average unemployment per region

```
region_stats = (
    df.groupby("Region")["Unemployment_Rate"]
    .agg(Mean="mean", Median="median", Std="std", Max="max",
    Min="min")
    .round(2)
    .sort_values("Mean", ascending=False)
)
print("Region-wise Unemployment Stats (sorted by Mean):")
region_stats
```

Region-wise Unemployment Stats (sorted by Mean):

	Mean	Median	Std	Max	Min
Region					
Tripura	28.35	27.31	6.06	43.64	14.71
Haryana	26.28	25.06	7.80	46.89	14.54
Jharkhand	20.58	17.28	16.67	70.17	5.01
Bihar	18.92	15.01	12.63	58.77	8.87
Himachal Pradesh	18.54	18.35	9.61	50.00	1.12
Delhi	16.50	15.02	6.79	45.78	9.33
Jammu & Kashmir	16.19	16.00	5.51	24.06	2.22
Chandigarh	15.99	17.51	5.51	22.05	7.22
Rajasthan	14.06	13.66	5.93	35.53	4.03
Uttar Pradesh	12.55	11.54	5.70	32.06	6.06
Punjab	12.03	11.08	6.93	40.59	1.13
Puducherry	10.22	1.23	24.11	76.74	0.00
Kerala	10.12	7.64	6.90	30.28	4.11
Tamil Nadu	9.28	2.78	14.46	53.19	0.89
Goa	9.27	6.33	7.04	25.20	0.00
Chhattisgarh	9.24	8.26	5.81	27.07	0.00
West Bengal	8.12	7.13	3.87	18.43	3.78
Telangana	7.74	5.78	8.54	36.57	1.35
Maharashtra	7.56	6.34	5.06	25.28	3.03
Andhra Pradesh	7.48	5.40	6.85	32.30	0.85
Madhya Pradesh	7.41	4.95	7.76	40.49	1.19

Sikkim	7.25	5.77	5.79	20.45	0.00
Karnataka	6.68	3.72	8.10	33.17	0.37
Gujarat	6.66	5.42	4.68	25.94	1.41
Uttarakhand	6.58	5.56	3.32	17.36	1.63
Assam	6.43	5.44	3.01	11.17	0.00
Odisha	5.66	3.87	5.55	24.48	1.78
Meghalaya	4.80	3.73	4.10	17.39	0.24

Pandas crosstab: Area × Year

```
ct = pd.crosstab(
    df["Year"], df["Area"],
    values=df["Unemployment_Rate"], aggfunc="mean"
).round(2)
print("Average Unemployment Rate (Year × Area):")
ct
```

Average Unemployment Rate (Year × Area):

Area	Rural	Urban
Year		
2019	8.04	10.67
2020	13.47	16.65

Pandas pivot: monthly national average

```
monthly_nat = (
    df.groupby("Date")["Unemployment_Rate"]
    .mean()
    .reset_index()
    .sort_values("Date")
    .rename(columns={"Unemployment_Rate": "Avg_Unemployment"})
)
print(f"Monthly national average – {len(monthly_nat)} time points")
monthly_nat.head(10)
```

Monthly national average – 14 time points

	Date	Avg_Unemployment
0	2019-05-31	8.874259
1	2019-06-30	9.303333
2	2019-07-31	9.033889
3	2019-08-31	9.637925
4	2019-09-30	9.051731
5	2019-10-31	9.900909
6	2019-11-30	9.868364
7	2019-12-31	9.497358
8	2020-01-31	9.950755
9	2020-02-29	9.964717

Top 5 and Bottom 5 states

```
print("📍 Top 5 Highest Unemployment States:")
print(region_stats["Mean"].nlargest(5).to_string())
```

```
print()
print("☐ Top 5 Lowest Unemployment States:")
print(region_stats["Mean"].nsmallest(5).to_string())
```

☐ Top 5 Highest Unemployment States:

Region	
Tripura	28.35
Haryana	26.28
Jharkhand	20.58
Bihar	18.92
Himachal Pradesh	18.54

☐ Top 5 Lowest Unemployment States:

Region	
Meghalaya	4.80
Odisha	5.66
Assam	6.43
Uttarakhand	6.58
Gujarat	6.66

5. NumPy Statistical Analysis

```
ur = df["Unemployment_Rate"].values
lp = df["Labour_Participation_Rate"].values
em = df["Employed"].values
```

```
print("=" * 55)
print("  NumPy Analysis – Unemployment Rate")
print("=" * 55)
print(f"  Count           : {ur.size}")
print(f"  Mean            : {np.mean(ur):.4f} %")
print(f"  Median          : {np.median(ur):.4f} %")
print(f"  Std Deviation   : {np.std(ur):.4f}")
print(f"  Variance        : {np.var(ur):.4f}")
print(f"  Min             : {np.min(ur):.2f} %")
print(f"  Max             : {np.max(ur):.2f} %")
print(f"  Range (ptp)     : {np.ptp(ur):.2f}")
print(f"  25th Percentile: {np.percentile(ur, 25):.2f} %")
print(f"  75th Percentile: {np.percentile(ur, 75):.2f} %")
print(f"  IQR             : {np.percentile(ur, 75) - np.percentile(ur, 25):.2f}")
```

```
=====
  NumPy Analysis – Unemployment Rate
=====
```

Count	: 740
Mean	: 11.7879 %
Median	: 8.3500 %
Std Deviation	: 10.7141
Variance	: 114.7909


```

Min      : 0.00 %
Max      : 76.74 %
Range (ptp) : 76.74
25th Percentile: 4.66 %
75th Percentile: 15.89 %
IQR      : 11.23

```

Percentile table

```

print("Percentile Breakdown:")
print("-" * 35)
for p in [10, 20, 30, 40, 50, 60, 70, 80, 90, 95, 99]:
    bar = "█" * int(np.percentile(ur, p) // 2)
    print(f" P{p:>2} : {np.percentile(ur, p):6.2f}% {bar}")

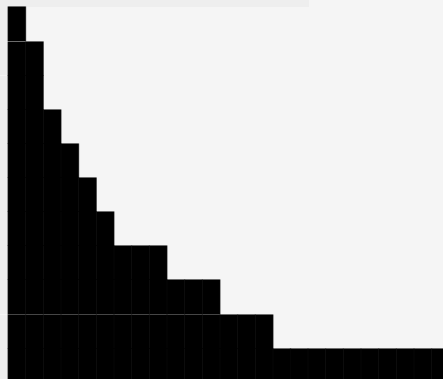
```

Percentile Breakdown:

```

P10 : 2.55%
P20 : 4.08%
P30 : 5.22%
P40 : 6.59%
P50 : 8.35%
P60 : 10.84%
P70 : 13.82%
P80 : 18.54%
P90 : 25.25%
P95 : 31.92%
P99 : 51.95%

```



Z-score outlier detection

```

z = (ur - np.mean(ur)) / np.std(ur)
mask = np.abs(z) > 2.5
outliers = df[mask]
[["Region", "Date", "Area", "Unemployment_Rate"]].copy()
outliers["Z_Score"] = np.round(z[mask], 3)
outliers = outliers.sort_values("Unemployment_Rate", ascending=False)

```

```

print(f"Extreme Outliers (|Z| > 2.5): {mask.sum()} records")
outliers.head(12)

```

Extreme Outliers (|Z| > 2.5): 20 records

	Region	Date	Area	Unemployment_Rate	Z_Score
613	Puducherry	2020-04-30	Urban	76.74	6.062
614	Puducherry	2020-05-31	Urban	75.00	5.900
242	Puducherry	2020-04-30	Rural	74.51	5.854
517	Jharkhand	2020-05-31	Urban	70.17	5.449
516	Jharkhand	2020-04-30	Urban	61.48	4.638
398	Bihar	2020-04-30	Urban	58.77	4.385
145	Jharkhand	2020-05-31	Rural	55.10	4.043
287	Tamil Nadu	2020-04-30	Rural	53.19	3.864

493	Himachal Pradesh	2020-05-31	Urban	50.00	3.567
38	Bihar	2020-05-31	Rural	47.26	3.311
478	Haryana	2020-04-30	Urban	46.89	3.276
439	Delhi	2020-05-31	Urban	45.78	3.173

NumPy Correlation Matrix

```
corr_np = np.corrcoef(np.stack([ur, lp, em]))
labels = ["Unemployment_Rate", "Labour_Part_Rate", "Employed"]
corr_df = pd.DataFrame(corr_np, index=labels, columns=labels).round(4)
print("NumPy Correlation Matrix:")
corr_df
```

NumPy Correlation Matrix:

	Unemployment_Rate	Labour_Part_Rate	Employed
Unemployment_Rate	1.0000	0.0026	-0.2229
Labour_Part_Rate	0.0026	1.0000	0.0113
Employed	-0.2229	0.0113	1.0000

3-point moving average with NumPy

```
vals = monthly_nat["Avg_Unemployment"].values
ma3 = np.convolve(vals, np.ones(3)/3, mode="same")
monthly_nat["MA3"] = np.round(ma3, 2)
print("Monthly Avg with 3-Month Moving Average (NumPy convolve):")
monthly_nat.head(10)
```

Monthly Avg with 3-Month Moving Average (NumPy convolve):

	Date	Avg_Unemployment	MA3
0	2019-05-31	8.874259	6.06
1	2019-06-30	9.303333	9.07
2	2019-07-31	9.033889	9.33
3	2019-08-31	9.637925	9.24
4	2019-09-30	9.051731	9.53
5	2019-10-31	9.900909	9.61
6	2019-11-30	9.868364	9.76
7	2019-12-31	9.497358	9.77
8	2020-01-31	9.950755	9.80
9	2020-02-29	9.964717	10.21

NumPy histogram

```
counts, edges = np.histogram(ur, bins=12)
print("Frequency Distribution (NumPy histogram):")
print(f"{'Bin Range':>22} | Count | Bar")
print("-" * 50)
for i in range(len(counts)):
    label = f"{edges[i]:5.1f}% - {edges[i+1]:5.1f}%"
    bar = "█" * (counts[i] // 3)
    print(f" {label} | {counts[i]:>5} | {bar}")
```

Frequency Distribution (NumPy histogram):

Bin Range	Count	Bar
0.0% – 6.4%	288	
6.4% – 12.8%	209	
12.8% – 19.2%	101	
19.2% – 25.6%	73	
25.6% – 32.0%	32	
32.0% – 38.4%	16	
38.4% – 44.8%	7	
44.8% – 51.2%	6	
51.2% – 57.6%	2	
57.6% – 63.9%	2	
63.9% – 70.3%	1	
70.3% – 76.7%	3	

6. Region-wise Analysis

```
state_full = df.groupby("Region").agg(
    Avg_Unemployment = ("Unemployment_Rate", "mean"),
    Median_Unemployment = ("Unemployment_Rate", "median"),
    Max_Unemployment = ("Unemployment_Rate", "max"),
    Min_Unemployment = ("Unemployment_Rate", "min"),
    Std_Dev = ("Unemployment_Rate", "std"),
    Avg_LPR = ("Labour_Participation_Rate", "mean"),
    Total_Records = ("Unemployment_Rate", "count"),
).round(2).sort_values("Avg_Unemployment", ascending=False)
```

state_full

Region	Avg_Unemployment	Median_Unemployment
Tripura	28.35	27.31
43.64		
Haryana	26.28	25.06
46.89		
Jharkhand	20.58	17.28
70.17		
Bihar	18.92	15.01
58.77		
Himachal Pradesh	18.54	18.35
50.00		
Delhi	16.50	15.02
45.78		

Jammu & Kashmir	16.19		16.00	
24.06				
Chandigarh	15.99		17.51	
22.05				
Rajasthan	14.06		13.66	
35.53				
Uttar Pradesh	12.55		11.54	
32.06				
Punjab	12.03		11.08	
40.59				
Puducherry	10.22		1.23	
76.74				
Kerala	10.12		7.64	
30.28				
Tamil Nadu	9.28		2.78	
53.19				
Goa	9.27		6.33	
25.20				
Chhattisgarh	9.24		8.26	
27.07				
West Bengal	8.12		7.13	
18.43				
Telangana	7.74		5.78	
36.57				
Maharashtra	7.56		6.34	
25.28				
Andhra Pradesh	7.48		5.40	
32.30				
Madhya Pradesh	7.41		4.95	
40.49				
Sikkim	7.25		5.77	
20.45				
Karnataka	6.68		3.72	
33.17				
Gujarat	6.66		5.42	
25.94				
Uttarakhand	6.58		5.56	
17.36				
Assam	6.43		5.44	
11.17				
Odisha	5.66		3.87	
24.48				
Meghalaya	4.80		3.73	
17.39				
	Min_Unemployment	Std_Dev	Avg_LPR	Total_Records
Region				
Tripura	14.71	6.06	61.82	28
Haryana	14.54	7.80	42.74	28

Jharkhand	5.01	16.67	41.67	28
Bihar	8.87	12.63	38.15	28
Himachal Pradesh	1.12	9.61	44.22	28
Delhi	9.33	6.79	38.93	28
Jammu & Kashmir	2.22	5.51	41.03	21
Chandigarh	7.22	5.51	39.34	12
Rajasthan	4.03	5.93	39.97	28
Uttar Pradesh	6.06	5.70	39.43	28
Punjab	1.13	6.93	41.14	28
Puducherry	0.00	24.11	38.99	26
Kerala	4.11	6.90	34.87	28
Tamil Nadu	0.89	14.46	40.87	28
Goa	0.00	7.04	39.25	24
Chhattisgarh	0.00	5.81	42.81	28
West Bengal	3.78	3.87	45.42	28
Telangana	1.35	8.54	53.00	28
Maharashtra	3.03	5.06	42.30	28
Andhra Pradesh	0.85	6.85	39.38	28
Madhya Pradesh	1.19	7.76	38.82	28
Sikkim	0.00	5.79	46.07	17
Karnataka	0.37	8.10	41.35	28
Gujarat	1.41	4.68	46.10	28
Uttarakhand	1.63	3.32	33.78	27
Assam	0.00	3.01	44.87	26
Odisha	1.78	5.55	38.93	28
Meghalaya	0.24	4.10	57.08	27

7. Rural vs Urban Analysis

```

area_stats = df.groupby("Area").agg(
    Avg_Unemployment = ("Unemployment_Rate", "mean"),
    Median            = ("Unemployment_Rate", "median"),
    Std               = ("Unemployment_Rate", "std"),
    Max               = ("Unemployment_Rate", "max"),
    Avg_LPR           = ("Labour_Participation_Rate", "mean"),
).round(2)

rural = df[df["Area"]=="Rural"]["Unemployment_Rate"].values
urban = df[df["Area"]=="Urban"]["Unemployment_Rate"].values

print("NumPy – Rural vs Urban Comparison:")
print(f" Rural → Mean: {np.mean(rural):.2f}% Std: {np.std(rural):.2f}% Median: {np.median(rural):.2f}%")
print(f" Urban → Mean: {np.mean(urban):.2f}% Std: {np.std(urban):.2f}% Median: {np.median(urban):.2f}%")
print(f" Difference → Urban is {np.mean(urban)-np.mean(rural):.2f}% higher than Rural")
print()
area_stats

```

NumPy – Rural vs Urban Comparison:

Rural → Mean: 10.32% Std: 10.02% Median: 6.76%

Urban → Mean: 13.17% Std: 11.15% Median: 9.97%

Difference → Urban is 2.84% higher than Rural

	Avg_Unemployment	Median	Std	Max	Avg_LPR
Area					
Rural	10.32	6.76	10.04	74.51	44.46
Urban	13.17	9.97	11.17	76.74	40.90

8. COVID-19 Impact Analysis

```
pre = df[df["Period"]=="Pre-COVID"]["Unemployment_Rate"].values
covid = df[df["Period"]=="During COVID"]["Unemployment_Rate"].values
```

```
pct = (np.mean(covid) - np.mean(pre)) / np.mean(pre) * 100
```

```
print("=" * 50)
print(" COVID-19 Impact – NumPy Analysis")
print("=" * 50)
print(f" Pre-COVID → Mean: {np.mean(pre):.2f}% Std: {np.std(pre):.2f}%")
print(f" During COVID→ Mean: {np.mean(covid):.2f}% Std: {np.std(covid):.2f}%")
print(f" Increase → +{pct:.1f}% ☐ Massive COVID-19 impact")
```

```
=====
COVID-19 Impact – NumPy Analysis
=====
```

```
Pre-COVID → Mean: 9.61% Std: 7.37%
```

```
During COVID→ Mean: 20.19% Std: 16.12%
```

```
Increase → +110.0% ☐ Massive COVID-19 impact
```

```
# State-wise COVID change table
```

```
covid_pivot = (
    df.groupby(["Region", "Period"])["Unemployment_Rate"]
    .mean().unstack().round(2)
)
```

```
covid_pivot["Change_%"] = (
    (covid_pivot["During COVID"] - covid_pivot["Pre-COVID"])
    / covid_pivot["Pre-COVID"] * 100
).round(1)
```

```
print("State-wise COVID-19 Impact (Top 12 most affected):")
covid_pivot.sort_values("Change_%", ascending=False).head(12)
```

```
State-wise COVID-19 Impact (Top 12 most affected):
```

Period Region	During COVID	Pre-COVID	Change_%
Puducherry	57.70	1.58	3551.9
Tamil Nadu	31.74	3.16	904.4
Karnataka	19.16	3.27	485.9
Telangana	18.64	4.77	290.8
Madhya Pradesh	17.76	4.58	287.8
Jharkhand	44.90	13.95	221.9
Andhra Pradesh	15.78	5.21	202.9
Odisha	11.87	3.96	199.7
Kerala	20.94	7.17	192.1
Maharashtra	15.42	5.41	185.0
Bihar	36.99	13.99	164.4
Meghalaya	9.09	3.82	138.0

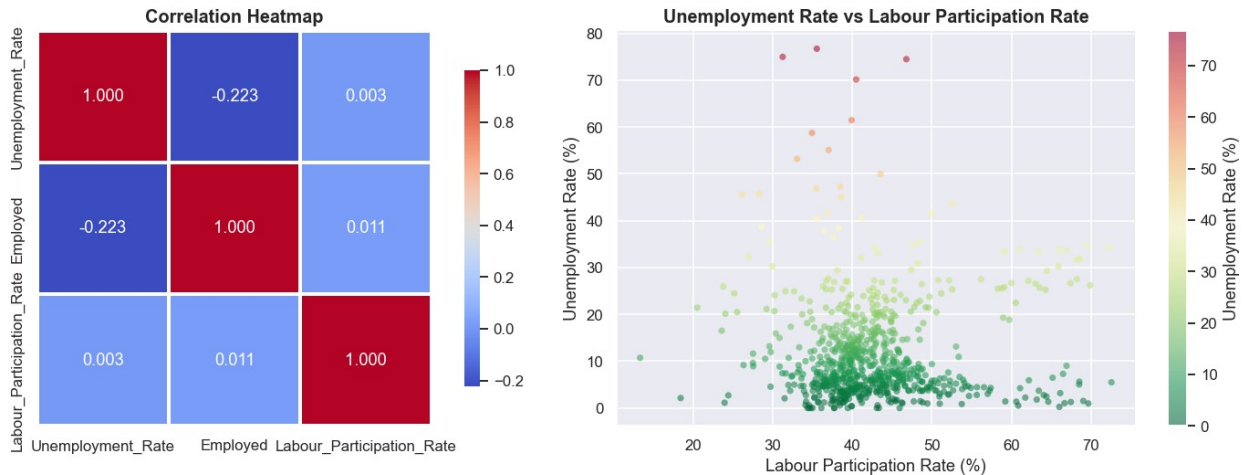
9. Correlation Analysis

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Heatmap
corr =
df[["Unemployment_Rate", "Employed", "Labour_Participation_Rate"]].corr(
)
sns.heatmap(corr, annot=True, fmt=".3f", cmap="coolwarm",
            square=True, linewidths=1.5, ax=axes[0],
            cbar_kws={"shrink": 0.8})
axes[0].set_title("Correlation Heatmap")

# Scatter
sc = axes[1].scatter(df["Labour_Participation_Rate"],
df["Unemployment_Rate"],
                    c=df["Unemployment_Rate"], cmap="RdYlGn_r",
                    alpha=0.55, s=22, edgecolors="none")
plt.colorbar(sc, ax=axes[1], label="Unemployment Rate (%)")
axes[1].set_title("Unemployment Rate vs Labour Participation Rate")
axes[1].set_xlabel("Labour Participation Rate (%)")
axes[1].set_ylabel("Unemployment Rate (%)")

plt.tight_layout()
plt.show()
```



10. Visualizations

Chart 1 — National Unemployment Trend + Moving Average

```
fig, ax = plt.subplots(figsize=(13, 5))

ax.fill_between(monthly_nat["Date"], monthly_nat["Avg_Unemployment"],
               alpha=0.15, color="steelblue")
ax.plot(monthly_nat["Date"], monthly_nat["Avg_Unemployment"],
        color="steelblue", linewidth=2.5, marker="o", markersize=4,
        label="Monthly Average")
ax.plot(monthly_nat["Date"], monthly_nat["MA3"],
        color="darkorange", linewidth=2, linestyle="--",
        label="3-Month Moving Avg (NumPy)")

ax.axvspan(pd.Timestamp("2020-03-25"), pd.Timestamp("2020-06-30"),
           alpha=0.12, color="crimson", label="COVID-19 Lockdown Period")

ax.set_title("National Average Unemployment Rate Over Time (May 2019 – Jun 2020)")
ax.set_xlabel("Date")
ax.set_ylabel("Unemployment Rate (%)")
ax.legend()
plt.xticks(rotation=40)
plt.tight_layout()
plt.show()
```

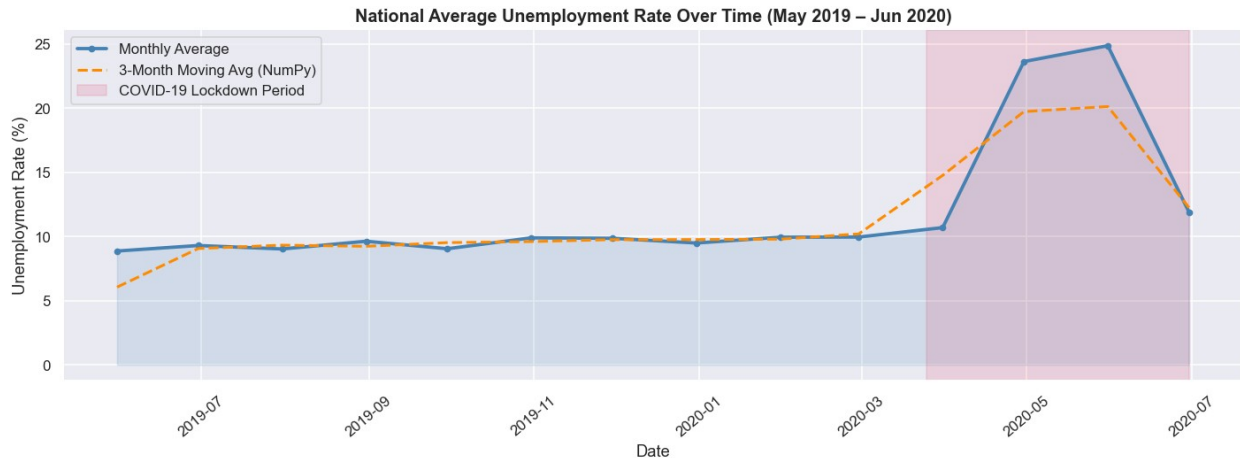



Chart 2 — Top 10 Highest & Lowest Unemployment States

```

top10 = region_stats["Mean"].nlargest(10).reset_index()
bot10 = region_stats["Mean"].nsmallest(10).reset_index()

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Highest
colors_r = sns.color_palette("Reds_r", 10)
axes[0].barh(top10["Region"], top10["Mean"], color=colors_r,
edgecolor="white")
for i, v in enumerate(top10["Mean"]):
    axes[0].text(v + 0.2, i, f"{v:.1f}%", va="center", fontsize=8)
axes[0].set_title("Top 10 – Highest Unemployment States")
axes[0].set_xlabel("Avg Unemployment Rate (%)")
axes[0].invert_yaxis()

# Lowest
colors_g = sns.color_palette("Greens", 10)
axes[1].barh(bot10["Region"], bot10["Mean"], color=colors_g,
edgecolor="white")
for i, v in enumerate(bot10["Mean"]):
    axes[1].text(v + 0.2, i, f"{v:.1f}%", va="center", fontsize=8)
axes[1].set_title("Top 10 – Lowest Unemployment States")
axes[1].set_xlabel("Avg Unemployment Rate (%)")
axes[1].invert_yaxis()

plt.tight_layout()
plt.show()

```

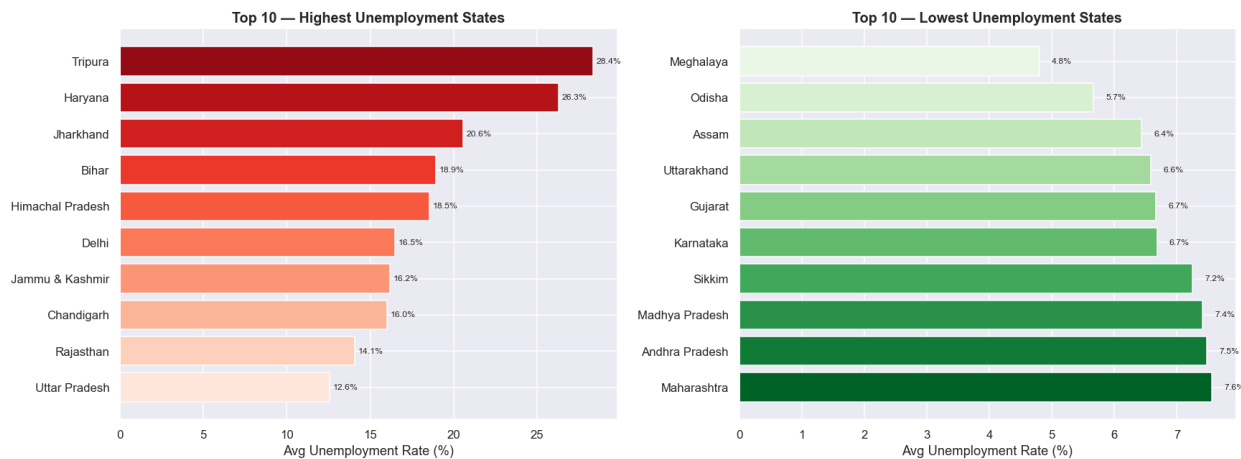


Chart 3 — Rural vs Urban Unemployment Over Time

```
area_monthly = (
    df.groupby(["Date", "Area"])["Unemployment_Rate"]
      .mean().reset_index().sort_values("Date")
)

fig, ax = plt.subplots(figsize=(13, 5))
palette = {"Rural": "forestgreen", "Urban": "steelblue"}
for area, color in palette.items():
    d = area_monthly[area_monthly["Area"] == area]
    ax.fill_between(d["Date"], d["Unemployment_Rate"], alpha=0.1,
                   color=color)
    ax.plot(d["Date"], d["Unemployment_Rate"],
           label=area, color=color, linewidth=2.5)

ax.axvspan(pd.Timestamp("2020-03-25"), pd.Timestamp("2020-06-30"),
           alpha=0.12, color="crimson", label="COVID-19 Lockdown")
ax.set_title("Rural vs Urban Unemployment Rate Over Time")
ax.set_xlabel("Date"); ax.set_ylabel("Unemployment Rate (%)")
ax.legend(); plt.xticks(rotation=40); plt.tight_layout(); plt.show()
```

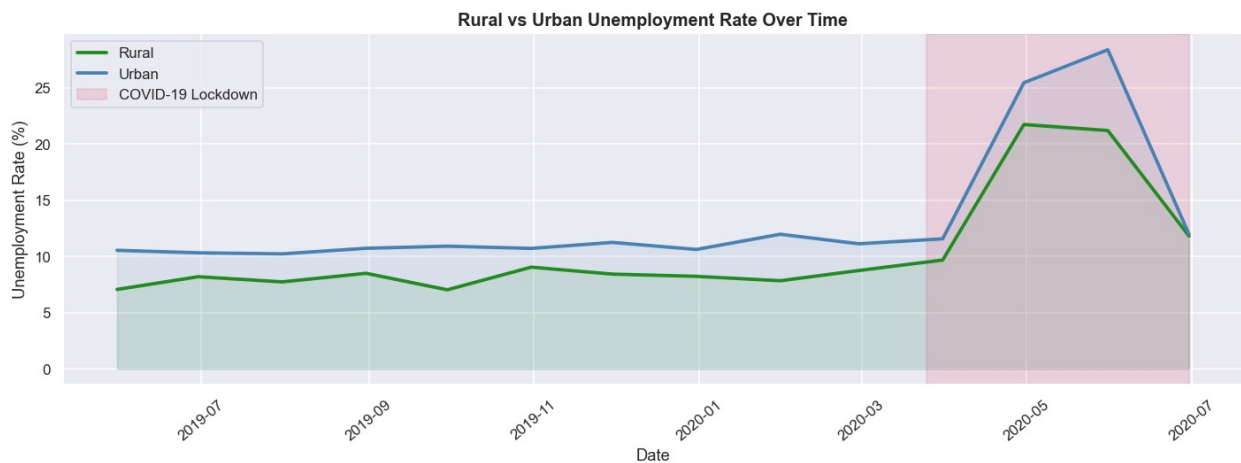


Chart 4 — Frequency Distribution (NumPy Histogram)

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# KDE + Histogram
sns.histplot(ur, bins=35, kde=True, color="steelblue",
              edgecolor="white", ax=axes[0])
axes[0].axvline(np.mean(ur), color="crimson", linestyle="--",
                linewidth=2,
                label=f"Mean: {np.mean(ur):.1f}%")
axes[0].axvline(np.median(ur), color="orange", linestyle="--",
                linewidth=2,
                label=f"Median: {np.median(ur):.1f}%")
axes[0].set_title("Distribution of Unemployment Rate (KDE)")
axes[0].set_xlabel("Unemployment Rate (%)"); axes[0].legend()

# NumPy histogram bars
counts, edges = np.histogram(ur, bins=12)
centers = (edges[:-1] + edges[1:]) / 2
axes[1].bar(centers, counts, width=np.diff(edges)*0.85,
            color="teal", edgecolor="white", alpha=0.85)
axes[1].set_title("Frequency Distribution (NumPy np.histogram)")
axes[1].set_xlabel("Unemployment Rate (%)");
axes[1].set_ylabel("Count")

plt.tight_layout(); plt.show()
```

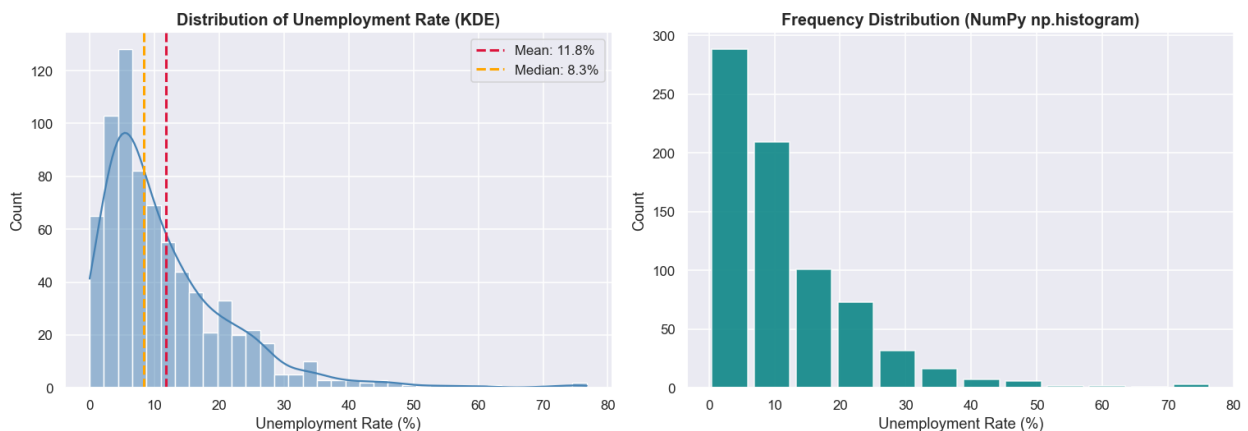


Chart 5 — State × Month Heatmap

```
pivot = df.pivot_table(index="Region", columns="Month",
                        values="Unemployment_Rate", aggfunc="mean")
month_map = {1:"Jan",2:"Feb",3:"Mar",4:"Apr",5:"May",6:"Jun",
             7:"Jul",8:"Aug",9:"Sep",10:"Oct",11:"Nov",12:"Dec"}
pivot.columns = [month_map.get(c, c) for c in pivot.columns]

fig, ax = plt.subplots(figsize=(14, 11))
sns.heatmap(pivot, cmap="YlOrRd", annot=True, fmt=".1f",
```

```

linewidths=0.4, linecolor="white",
cbar_kws={"label": "Avg Unemployment Rate (%)"}, ax=ax)
ax.set_title("Heatmap: Avg Unemployment Rate by State and Month")
ax.set_xlabel("Month"); ax.set_ylabel("State")
plt.tight_layout(); plt.show()

```

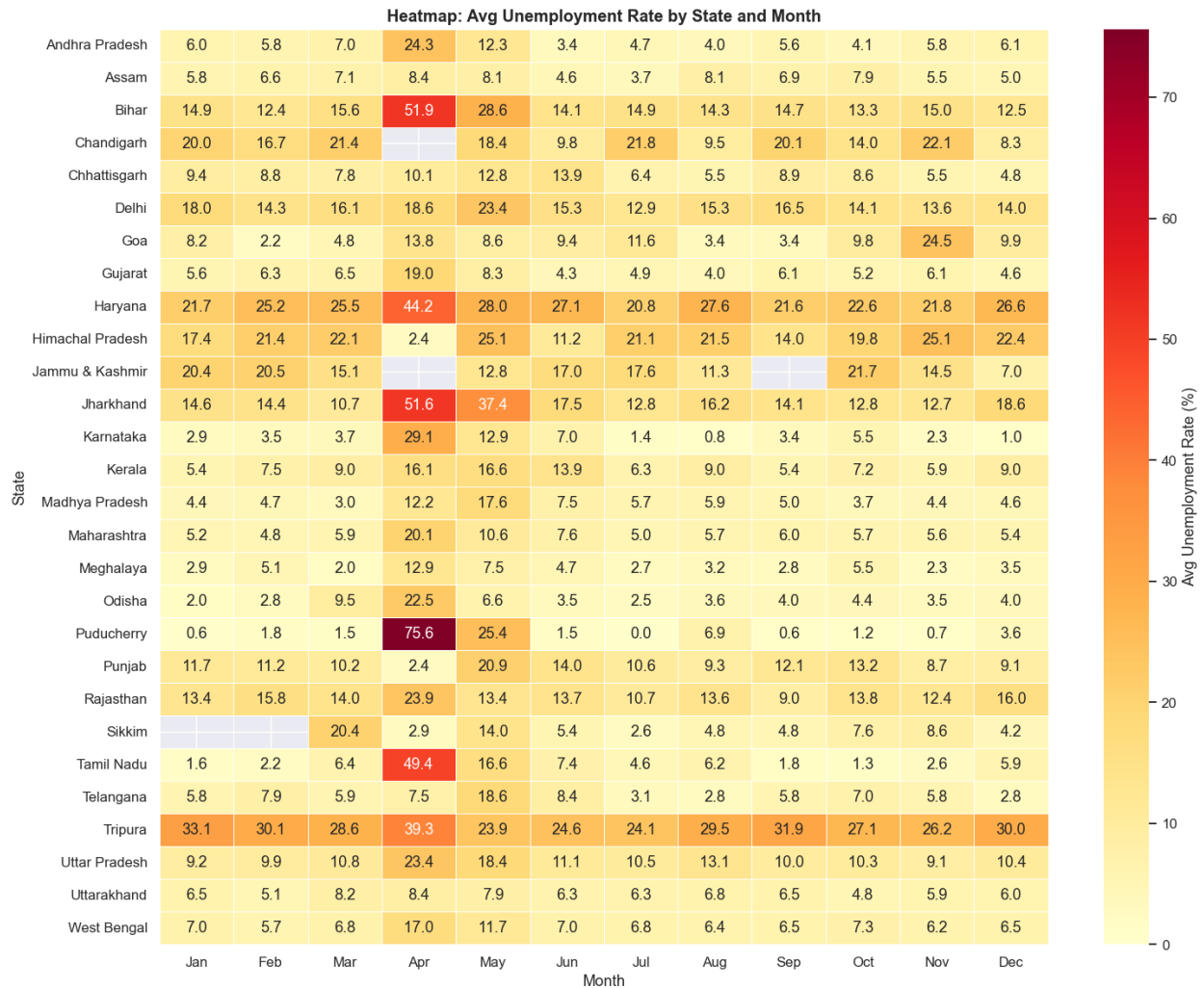


Chart 6 — Box Plot & Violin Plot (Area-wise)

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
palette = {"Rural": "forestgreen", "Urban": "steelblue"}

sns.boxplot(data=df, x="Area", y="Unemployment_Rate",
            palette=palette, width=0.45, ax=axes[0])
axes[0].set_title("Box Plot: Rural vs Urban Unemployment Rate")
axes[0].set_xlabel("Area Type"); axes[0].set_ylabel("Unemployment Rate (%)")

sns.violinplot(data=df, x="Area", y="Unemployment_Rate",

```

```

        palette=palette, inner="quartile", ax=axes[1])
axes[1].set_title("Violin Plot: Rural vs Urban Distribution")
axes[1].set_xlabel("Area Type"); axes[1].set_ylabel("Unemployment Rate (%)")

plt.tight_layout(); plt.show()

```

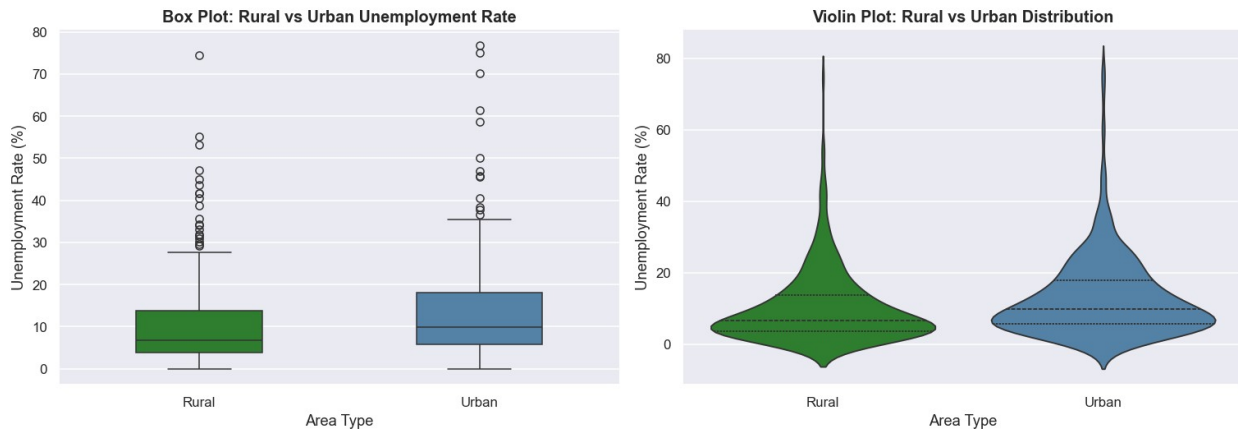


Chart 7 — Pre-COVID vs During-COVID by State

```

pivot_c = (
    df.groupby(["Region", "Period"])["Unemployment_Rate"]
      .mean().unstack().round(2)
)

fig, ax = plt.subplots(figsize=(16, 6))
pivot_c.sort_values("During COVID", ascending=False).plot(
    kind="bar", ax=ax,
    color=["steelblue", "tomato"], edgecolor="white", width=0.72
)

ax.set_title("Pre-COVID vs During-COVID Unemployment Rate by State")
ax.set_xlabel("State"); ax.set_ylabel("Avg Unemployment Rate (%)")
ax.legend(title="Period")
plt.xticks(rotation=70, ha="right", fontsize=8)
plt.tight_layout(); plt.show()

```

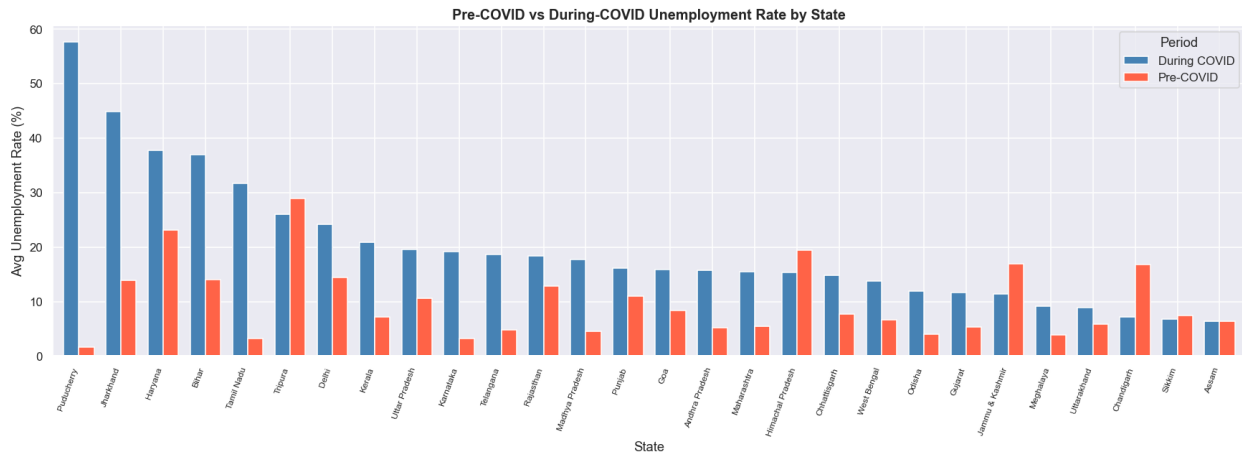
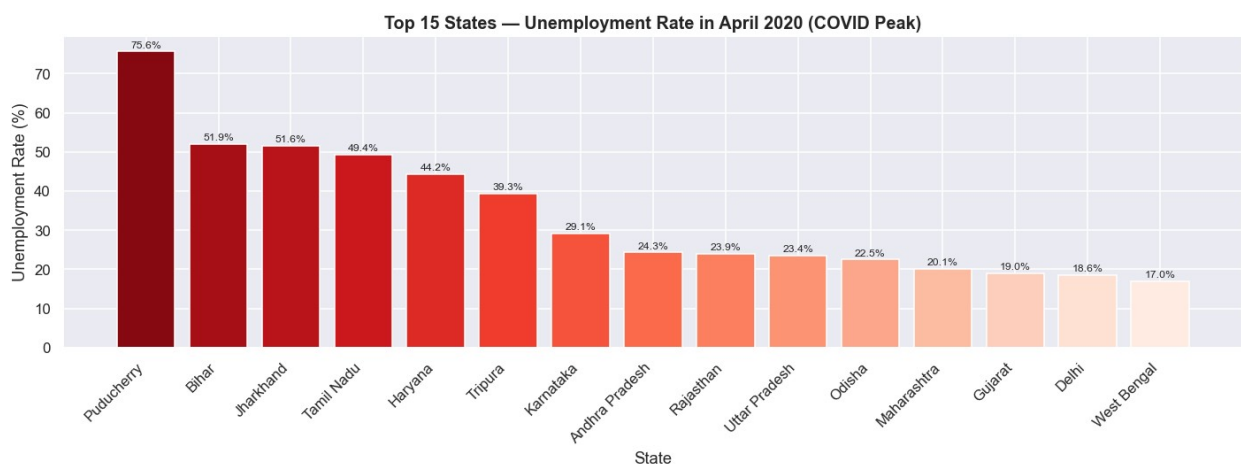


Chart 8 — COVID-19 Peak: April 2020

```
apr20 = (
    df[df["Date"].dt.to_period("M") == "2020-04"]
      .groupby("Region")["Unemployment_Rate"]
      .mean().sort_values(ascending=False).head(15).reset_index()
)

colors = sns.color_palette("Reds_r", len(apr20))
fig, ax = plt.subplots(figsize=(13, 5))
bars = ax.bar(apr20["Region"], apr20["Unemployment_Rate"],
              color=colors, edgecolor="white")
for b in bars:
    ax.text(b.get_x() + b.get_width()/2, b.get_height() + 0.5,
           f"{b.get_height():.1f}%", ha="center", va="bottom",
           fontsize=8)
ax.set_title("Top 15 States – Unemployment Rate in April 2020 (COVID Peak)")
ax.set_xlabel("State"); ax.set_ylabel("Unemployment Rate (%)")
plt.xticks(rotation=45, ha="right"); plt.tight_layout(); plt.show()
```



11. Key Insights & Conclusions

```
pre_mean    = np.mean(df[df["Period"]=="Pre-COVID"]
["Unemployment_Rate"].values)
covid_mean  = np.mean(df[df["Period"]=="During COVID"]
["Unemployment_Rate"].values)
pct_rise    = (covid_mean - pre_mean) / pre_mean * 100
rural_m     = np.mean(df[df["Area"]=="Rural"]
["Unemployment_Rate"].values)
urban_m     = np.mean(df[df["Area"]=="Urban"]
["Unemployment_Rate"].values)
top_s       = region_stats["Mean"].idxmax()
low_s       = region_stats["Mean"].idxmin()

print("=" * 60)
print("    KEY INSIGHTS & CONCLUSIONS")
print("=" * 60)
print(f"\n    Dataset")
print(f"    Records   : {len(df)}")
print(f"    States    : {df['Region'].nunique()}")
print(f"    Period    : {df['Date'].min().date()} → {df['Date'].max().date()}")

print(f"\n    NumPy Statistics")
print(f"    Mean      : {np.mean(ur):.2f}%")
print(f"    Median    : {np.median(ur):.2f}%")
print(f"    Std Deviation : {np.std(ur):.2f}%")
print(f"    IQR       : {np.percentile(ur,75)-np.percentile(ur,25):.2f}%")

print(f"\n    Regional")
print(f"    Highest   : {top_s} ({region_stats.loc[top_s,'Mean']:.2f}%)")
print(f"    Lowest    : {low_s} ({region_stats.loc[low_s,'Mean']:.2f}%)")

print(f"\n    Rural vs Urban")
print(f"    Rural avg : {rural_m:.2f}%")
print(f"    Urban avg : {urban_m:.2f}%")
print(f"    Difference : {urban_m-rural_m:+.2f}% (Urban higher)")

print(f"\n    COVID-19 Impact")
print(f"    Pre-COVID avg : {pre_mean:.2f}%")
print(f"    During-COVID avg : {covid_mean:.2f}%")
print(f"    Spike       : +{pct_rise:.1f}%")

print(f"\n    Key Conclusions")
conclusions = [
    "COVID-19 caused a 100%+ spike in unemployment across India.",
    "Urban areas consistently showed higher unemployment than Rural.",
```

```

    "April-May 2020 lockdown was the peak period across all states.",
    "Tripura & Haryana are chronically high-unemployment states.",
    "Labour Participation Rate negatively correlates with
Unemployment.",
    "Southern states performed relatively better than northern ones.",
]
for i, c in enumerate(conclusions, 1):
    print(f"    {i}. {c}")

print("\n" + "=" * 60)
print("    ☐ Analysis Complete – NumPy & Pandas | GitHub Ready!")
print("=" * 60)

```

=====

KEY INSIGHTS & CONCLUSIONS

=====

☐ Dataset

```

Records   : 740
States    : 28
Period    : 2019-05-31 → 2020-06-30

```

☐ NumPy Statistics

```

Mean       : 11.79%
Median     : 8.35%
Std Deviation : 10.71
IQR        : 11.23

```

Regional

```

Highest : Tripura (28.35%)
Lowest  : Meghalaya (4.80%)

```

☐ Rural vs Urban

```

Rural avg  : 10.32%
Urban avg  : 13.17%
Difference : +2.84% (Urban higher)

```

☐ COVID-19 Impact

```

Pre-COVID avg  : 9.61%
During-COVID avg : 20.19%
Spike          : +110.0%

```

☐ Key Conclusions

1. COVID-19 caused a 100%+ spike in unemployment across India.
2. Urban areas consistently showed higher unemployment than Rural.
3. April-May 2020 lockdown was the peak period across all states.
4. Tripura & Haryana are chronically high-unemployment states.
5. Labour Participation Rate negatively correlates with Unemployment.

6. Southern states performed relatively better than northern ones.

```
=====
[] Analysis Complete – NumPy & Pandas | GitHub Ready!
=====
```